

Deep Reinforcement Learning for Dynamic Learn to Rank: A Risk-Aware Framework for Cryptocurrency

Sasan Barak^{1,*}
Alireza Mousavi¹
Seyed Ali Hosseini²

¹ Department of Decision Analytics and Risk, Southampton Business School, University of Southampton, UK

² Department of Energy, Polytechnic University of Milan, Milan, Italy

* Corresponding Author: Sasan Barak

Department of Decision Analytics and Risk, Southampton Business School, University of Southampton, UK

Building 175, Boldrewood Innovation Campus, Burgess Rd, Southampton, UK. SO16 7QF

Email: S.Barak@soton.ac.uk

Deep Reinforcement Learning for Dynamic Learn to Rank: A Risk-Aware Framework for Cryptocurrency

Abstract

This paper applies deep reinforcement learning (DRL) to cross-sectional asset allocation, reformulating the task as a sequential decision problem to address the limitations of static ranking models in volatile cryptocurrency markets. The static nature of traditional quantitative strategies and contemporary Learning-to-Rank (LTR) models—which treat each rebalancing decision as an isolated problem—can be ineffective in financial markets, leading to highly negative returns and notable crash risk. To address this, we develop a modular, three-stage framework composed of a DRL agent that learns a dynamic ranking policy, a supervised meta-learning filter that gates trades based on volatility forecasts, and risk-based portfolio optimizers for allocation. Tested empirically on 12-hour cryptocurrency data from 2020 to 2024 using a walk-forward validation protocol, our dynamic approach outperforms the static benchmarks, many of which generated negative returns. The final integrated framework achieves a Sharpe ratio of 2.85, which remains high after accounting for transaction costs. Behavioral analysis indicates the source of this performance: the agent learns a regime-adaptive policy, employing contrarian strategies in low-volatility periods and switching to momentum-based logic during market stress, providing empirical support for the adaptive markets hypothesis.

Keywords: Finance; Deep Reinforcement Learning; Learning to Rank; Portfolio Optimization; Sequential Decision-Making.

1 Introduction

The ability to accurately rank assets based on expected future returns remains fundamental to cross-sectional investment strategies, yet traditional approaches face critical limitations in volatile, non-stationary markets. While momentum and other cross-sectional strategies have generated persistent risk premia in equity markets (Jegadeesh & Titman, 1993), their direct application to cryptocurrency markets—characterized by extreme volatility clustering (Katsiampa, 2017), unstable correlation structures (Borri, 2019), and continuous trading across fragmented exchanges (Makarov & Schoar, 2020)—often results in catastrophic performance with significant crash risk (Y. Liu & Tsyvinski, 2021).

Recent advances in learning-to-rank (LTR) methodologies have shown promise in addressing the ranking challenges inherent to cross-sectional strategies. By explicitly optimizing ordinal structures rather than point estimates, LTR algorithms such as RankNet, ListNet, and LambdaMART have demonstrated superior performance over traditional heuristics and regression-based approaches (Poh et al., 2021, 2022). These methods effectively capture pairwise and listwise relationships between assets, leading to more accurate rankings and improved risk-adjusted returns. However, a fundamental limitation persists: existing LTR approaches remain inherently

static, treating each rebalancing decision as an isolated optimization problem while ignoring the sequential, path-dependent nature of portfolio management.

This static limitation proves particularly damaging in cryptocurrency markets, where regime shifts occur frequently and correlation structures break down during periods of stress. A profitable signal today may position a portfolio vulnerably to a volatility shock hours later—a path dependency that static models cannot anticipate or mitigate. The inability to internalize how current decisions affect future risk and opportunity represents a critical gap between the sequential reality of portfolio management and the single-period optimization assumptions underlying existing LTR frameworks. The cryptocurrency market, with its high-frequency data and evolving microstructure (Vidal-Tomás, 2021), thus serves as an ideal laboratory for testing the limits of static models and the potential of dynamic, adaptive frameworks.

We address this fundamental limitation by reformulating the cross-sectional asset allocation problem as a sequential decision-making task under uncertainty, operationalized through deep reinforcement learning (DRL). By modeling portfolio construction as a Markov decision process, our DRL agent learns a dynamic policy that maps evolving market states to optimal rankings while explicitly accounting for the long-horizon consequences of current actions. This approach aligns with the adaptive markets hypothesis (Lo, 2004), which posits that market efficiency evolves over time and successful strategies must adapt to changing market regimes rather than assume stationarity.

While DRL provides a powerful theoretical solution, its practical implementation for institutional asset management requires a transparent and controllable architecture. Our framework is therefore built from three modular components designed to address these adoption concerns. First, a DRL signal generator produces regime-adaptive rankings through sequential learning. Second, a meta-learning risk filter dynamically gates trade execution by using a supervised model trained on the strategy’s own recent return patterns to forecast periods where its near-term volatility is likely to exceed a specified threshold. Third, risk-based portfolio optimizers translate rankings into allocations, decoupling alpha generation from risk management to avoid the "black box" critique of end-to-end systems.

Empirically testing on 12-hour cryptocurrency data from 2020 to 2024 using rigorous walk-forward validation, our results provide compelling evidence for the superiority of dynamic approaches. The DRL strategy dramatically outperforming all static benchmarks—including state-of-the-art LTR models and classical momentum strategies—which uniformly generate substantial losses. Behavioral analysis reveals the source of this outperformance: the agent learns a sophisticated, regime-adaptive policy that employs contrarian mean-reversion during low-volatility periods while switching to momentum strategies during market stress, providing novel empirical support for adaptive market dynamics.

The integrated framework, combining DRL rankings with meta-learning filters and maximum diversification allocation (Choueifaty & Coignard, 2008), achieves a final Sharpe ratio of 2.85, robust to transaction costs and parameter variations. This performance demonstrates the synergistic interaction between data-driven signals and modern portfolio construction techniques—a relationship that remains underexplored in the literature.

This study contributes to the literature in several ways. First, we pioneer the application

of DRL to cross-sectional ranking, demonstrating how sequential optimization uncovers regime-adaptive policies that static models cannot replicate, providing novel evidence supporting the adaptive markets hypothesis. Second, our modular architecture addresses institutional barriers to AI adoption by separating signal generation from risk management. Third, we systematically analyze the interaction between data-driven ranking signals and portfolio construction methods, documenting powerful synergies between DRL signals and risk-based allocation techniques.

The remainder of the paper proceeds as follows. Section 2 reviews related literature on cross-sectional strategies, reinforcement learning, and adaptive markets. Section 3 details our DRL framework, meta-learning filter, and portfolio construction methodology. Section 4 presents empirical results and behavioral analysis. Section 5 discusses robustness and limitations. Finally, Section 6 concludes by considering the broader implications of our framework for other volatile asset classes.

2 Literature Review

This section synthesizes five interconnected streams of literature to build a compelling case for our methodological framework: (1) cryptocurrency market characteristics, (2) the evolution of cross-sectional strategies, (3) the application of reinforcement learning in finance, (4) market regime analysis and adaptive strategies, and (5) modern portfolio construction. By critically evaluating each stream, we identify specific, unaddressed challenges that motivate our integrated approach.

2.1 Cryptocurrency Market Foundations and Anomalies

Cryptocurrency markets present an environment fundamentally distinct from traditional finance. Their decentralized market structure, evidenced by the persistent cross-exchange price discrepancies documented by Makarov and Schoar (2020), challenges classical notions of price discovery. Statistically, the market is defined by extreme and asymmetric volatility patterns, which demand more sophisticated modeling than standard GARCH frameworks (Katsiampa, 2017; Phillip et al., 2018). While a strong momentum premium exists, Y. Liu and Tsyvinski (2021)’s analysis reveals it is accompanied by substantial crash risk. This danger is compounded by an unstable correlation structure, where diversification benefits evaporate during periods of market stress (Borri, 2019). Taken together, these characteristics create an environment defined by extreme tail risk for traditional quantitative strategies, demanding novel approaches that are inherently adaptive and risk-aware.

2.2 The Evolution of Cross-Sectional Investment Strategies

Our work builds on the foundational literature of cross-sectional momentum, which Jegadeesh and Titman (1993) first documented in equities. However, this strategy exhibits severe crash risk during stress periods (Daniel & Moskowitz, 2016) and performs poorly in volatile markets such as cryptocurrencies (Y. Liu & Tsyvinski, 2021), particularly when its underlying linear assumptions break down during regime shifts (Daniel & Moskowitz, 2016). While modern approaches improve upon this by using nonlinear machine learning models for return prediction (Gu et al., 2020), they

typically follow a flawed ‘regress-then-rank’ methodology. This approach is suboptimal because it trains models on a pointwise loss function to optimize for individual point estimates (i.e., return forecasts), but fundamentally ignores the relative ordering of assets required for portfolio construction. Learning-to-Rank (LTR) algorithms represent a direct solution to this mismatch. By explicitly optimizing for ordinal structures, the LTR directly targets the ranking objective at the core of the investment process (Poh et al., 2021, 2022).

The primary innovation of LTR algorithms is their departure from treating asset selection as a simple regression problem. Instead, they are categorized by how they process relational information through their loss functions (T.-Y. Liu, 2011). Pointwise approaches represent the simplest method, framing the task as a regression or classification problem for individual assets, thereby ignoring the group structure and relative information between securities. Pairwise methods, such as the influential RankNet model (C. Burges et al., 2005; C. J. Burges, 2010), offer a significant improvement by recasting the problem as a classification of asset pairs, with the learning objective of minimizing the number of incorrectly ordered pairs. The most sophisticated methods are listwise approaches (e.g., ListNet by Cao et al. 2007; ListMLE by Xia et al. 2008), which consider the entire list of assets when making predictions. By directly optimizing a loss function defined on the whole permutation of assets, these models more closely align the training objective with the ultimate goal of producing an optimal ranked list.

Despite their sophistication, these foundational LTR models are typically “globally learned”, a characteristic that creates a critical weakness in dynamic financial markets (Poh et al., 2022). While interactions between assets are considered in the loss function during training, the resulting model scores each asset individually at inference time, ignoring the immediate context of other assets being ranked. This flaw makes the models susceptible to producing suboptimal rankings during specific market regimes, such as risk-off episodes. To address this, a more advanced branch of LTR research focuses on re-ranking using the “local ranking context”—the feature distributions of the top and bottom-ranked assets—to refine an initial list. Recent work in this area has moved from using Recurrent Neural Networks (RNNs) to capture this context to more powerful Transformer-based architectures that leverage self-attention to model the complex, non-sequential inter-item dependencies more effectively (Poh et al., 2022).

However, a crucial shortcoming of this entire stream of research—including these context-aware models—is its static perspective. These LTR methods treat each ranking decision as an isolated optimization problem, fundamentally ignoring the path-dependent and sequential nature of portfolio management, where today’s trades directly impact future risk and opportunities.

2.3 Reinforcement Learning in Financial Decision Making

Reinforcement learning (RL) offers a powerful paradigm to overcome the static limitations of supervised learning. By framing trading as a Markov Decision Process (MDP), an RL agent learns a policy to optimize a cumulative reward over a long horizon (Sutton & Barto, 2018). Within the finance literature, two distinct approaches for applying RL have emerged: end-to-end frameworks where agents directly output portfolio weights, and decoupled frameworks where RL generates signals or rankings that feed into separate portfolio construction modules (Hambly et al., 2023). Each approach presents distinct trade-offs between theoretical elegance and practical

implementation concerns.

The first approach involves end-to-end frameworks where the agent’s actions directly correspond to portfolio weights. Seminal work by Jiang et al. (2017) established the feasibility of this approach. While theoretically elegant, it carries a significant practical drawback: the direct output of weights can function as an uninterpretable "black box," making risk attribution, constraint enforcement, and diagnosis exceptionally difficult for institutional asset managers (Rudin, 2019).

The second approach involves decoupled frameworks that separate signal generation from portfolio construction. In this paradigm, RL agents learn to rank or score assets, while portfolio optimization remains a distinct module. This architecture offers greater transparency and modularity, allowing practitioners to integrate novel alpha sources within existing risk management frameworks. This decoupled approach has gained traction in complex trading tasks such as optimal limit order execution (Schnaubelt, 2022).

2.4 Regime-Aware Filtering and Adaptive Strategies

A central theme in cryptocurrency trading literature is the need for adaptive strategies, a notion supported by extensive research on regime-switching models (Ang & Bekaert, 2002; Ardia et al., 2019; Hamilton, 1989). The most common adaptation mechanism involves volatility-timing rules, whose effectiveness in traditional equity markets is well-documented (Daniel & Moskowitz, 2016; Moreira & Muir, 2017). These approaches typically scale position sizes or filter trading signals based on realized volatility measures.

However, recent advances in meta-learning approaches offer more sophisticated mechanisms. Joubert (2022) introduce meta-labeling as a machine learning layer that sits on top of base strategies to size positions and filter false-positive signals, demonstrating improvements in Sharpe ratios and drawdown control. Meyer et al. (2023) extend this framework by investigating probability calibration techniques for position sizing, showing that meta-models can significantly outperform fixed-size approaches. These studies highlight the potential for multi-layered, adaptive frameworks that can learn complex mappings from market states to trading decisions.

Despite these advances, existing approaches mainly focus on traditional asset classes. Cryptocurrency markets exhibit unique characteristics—extreme volatility clustering, regime-dependent cross-sectional dispersions, and complex volume-return relationships—that may require fundamentally different adaptation mechanisms than those developed for equity markets.

2.5 Portfolio Construction Beyond Mean-Variance

The final step in any investment process is portfolio construction. The pitfalls of traditional mean-variance optimization (MVO) are well-known; Michaud (1989)’s famous "error maximization" critique highlights its extreme sensitivity to estimation errors. This finding was powerfully reinforced by the comprehensive analysis of DeMiguel et al. (2009), which showed that few sophisticated strategies reliably outperform the naive 1/N benchmark.

In response to these limitations, the literature has developed numerous alternative approaches. Risk-based allocation methods, which operate independently of return forecasts, have gained particular traction. Maillard et al. (2010) demonstrate the theoretical and empirical benefits of Risk

Parity approaches, while Choueifaty and Coignard (2008) introduce Maximum Diversification as a method to maximize the diversification ratio. Similarly, Rockafellar and Uryasev (2000) establish CVaR optimization as a robust alternative that directly addresses tail risk concerns.

The rise of machine learning introduces a critical, yet under-explored, dimension to this problem. While ML models may produce superior return predictions, they also generate more complex and potentially unstable forecasts that could exacerbate MVO’s inherent weaknesses. This intersection between sophisticated signal generation and robust portfolio construction remains an active area of research, particularly in volatile asset classes where traditional optimization methods face additional challenges.

3 Methodology

This section details the integrated framework designed to learn and execute dynamic ranking. Our approach is based on an adaptable, three-stage architecture aimed at systematically tackling the specific difficulties of ranking, risk management, and capital allocation in a highly non-stationary environment. By deliberately departing from opaque, end-to-end systems, this breakdown improves transparency, diagnostic capability, and adaptability. The framework consists of: (1) a Deep Reinforcement Learning (DRL) agent for dynamic asset ranking, formulated as a sequential decision problem; (2) a supervised meta-learning filter for adaptive, volatility-driven risk management; and (3) a robust portfolio construction module for systematic capital allocation. We begin by formalizing the problem through the transition from conventional cross-sectional models to a more appropriate Markov Decision Process (Section 3.1). Afterwards, we delineate the DRL architecture and training process (Section 3.2), present the meta-learning risk filter (Section 3.3), and conclude with a formal investigation into portfolio construction methods (Section 3.4).

Table 1: Primary Notation

Symbol	Definition	Symbol	Definition
General			
N	Number of assets	H_p	Holding period for trades
t, T	Time step, Episode horizon	T_{total}	Total backtest steps
H	Lookback window	K	Assets per portfolio leg
MDP Formulation & DRL Agent			
S, s_t	State space, State at time t	π_θ	Actor’s policy
A, a_t	Action space, Action at time t	V_ψ	Critic’s value function
R, r_{k+1}	Reward function, Reward	G_t^n	n -step return target
γ	Discount factor	A_t	Advantage estimate
π, π^*	Policy, Optimal policy	β	Entropy coefficient
Meta-Learning Filter			
W_{vol}	Lookback for volatility	W_{feat}	Lookback for features
σ_t	Rolling strategy volatility	τ	Volatility threshold
y_t^{vol}	Binary volatility label		
Portfolio Construction			
T_{hist}	Historical scenarios for CVaR		
w, w_i	Portfolio weights vector, Weight of asset i	σ	Vector of asset volatilities
Σ^{LW}	Ledoit-Wolf covariance matrix	ζ, α	VaR and confidence level
Performance Metrics			
r_p	Portfolio return series	θ	Omega Ratio return threshold

3.1 Problem Formulation: From Static Ranking to a Dynamic Policy

3.1.1 Cross-Sectional Momentum Strategy Framework

The foundational approach to cross-sectional momentum, established in seminal works such as Jegadeesh and Titman (1993), provides a crucial theoretical baseline. This framework operates via a static, four-step process at each discrete rebalancing period $t \in \{1, 2, \dots, T_{total}\}$ across a universe of N assets:

Score calculation. For each asset i , a predictive model f computes a ranking score Y_t^i based on a feature vector u_t^i :

$$Y_t^i = f(u_t^i) \quad \forall i \in \{1, 2, \dots, N\} \quad (1)$$

Ordinal Ranking. The continuous scores are mapped to ordinal ranks Z_t^i via a ranking operator $\mathcal{R}(\cdot)$, where $Z_t^i \in \{1, 2, \dots, N\}$.

Security Selection. Portfolios are constructed by selecting the top K and bottom K assets from the rank distribution. The trading signal X_t^i for asset i is defined as:

$$X_t^i = \begin{cases} +1 & \text{if asset } i \text{ is in the top } K \\ -1 & \text{if asset } i \text{ is in the bottom } K \\ 0 & \text{otherwise} \end{cases} \quad (2)$$

Portfolio Construction. The cash-neutral portfolio return over $[t, t + 1]$ is computed, typically under an equal-weighting scheme:

$$r_{t,t+1}^p = \frac{1}{2} \left(\frac{1}{n_t^L} \sum_{i \in \mathcal{L}_t} r_{t+1}^i - \frac{1}{n_t^S} \sum_{i \in \mathcal{S}_t} r_{t+1}^i \right) \quad (3)$$

where $\mathcal{L}_t$ and $\mathcal{S}_t$ are the long and short portfolios of size n_t^L and n_t^S , and r_{t+1}^i is the log-return of asset i .

This “predict, then optimize” paradigm, while foundational, is inherently limited by its static, path-independent nature. It treats each rebalancing decision as an isolated event, failing to account for how current actions influence future state transitions and opportunities, a critical deficiency in markets characterized by volatility clustering and unstable correlation matrices.

3.1.2 A Dynamic Alternative: The Markov Decision Process Formulation

To overcome these limitations, we model the dynamic asset ranking problem as a finite-horizon Markov Decision Process (MDP), defined by the tuple $\mathcal{M} = \langle S, A, R, S' \rangle$, where S , A , R , and S' denote the state space, action space, reward function, and next state, respectively. This formulation allows an agent to learn an optimal policy π^* through direct interaction with the market environment, thereby optimizing a sequence of decisions over time (Sutton & Barto, 2018). Figure 1 illustrates the MDP components and the agent’s decision-making process.

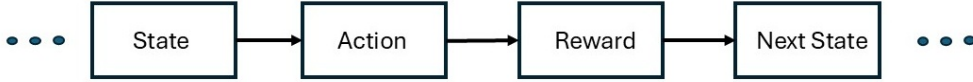


Figure 1: Overview of the Markov Decision Process (MDP) structure

State Space (S). The state $s_t \in S$ at decision epoch t must encode enough market information to reasonably approximate the Markov property. In our implementation, the state is defined as a tensor of historical closing prices over a lookback window H :

$$s_t = \{u_{t-H+1}, u_{t-H+2}, \dots, u_t\} \in \mathbb{R}^{N \times H} \quad (4)$$

where $u_t \in \mathbb{R}^N$ represents the vector of normalized closing prices for N assets at time t . We use closing prices alone as a deliberate simplification for three reasons. First, they are the most consistently available and standardized data across all assets, ensuring uniformity. Second, this minimal input reduces model complexity, which helps mitigate overfitting and enables more efficient training. Finally, using only price history establishes a controlled baseline, allowing future studies to systematically evaluate the added value of incorporating other features such as volume, volatility, or sentiment through ablation or feature augmentation experiments.

Action space (A). At each time t , the agent selects an action $a_t \in A$, defined as a vector of real-valued scores for all N assets:

$$a_t = [a_{t,1}, a_{t,2}, \dots, a_{t,N}]^\top \in \mathbb{R}^N \quad (5)$$

These continuous scores are subsequently transformed into ordinal rankings for portfolio construction. This design choice deliberately decouples the agent’s policy (ranking) from the final capital allocation, enhancing transparency and modularity compared to end-to-end approaches that directly output portfolio weights.

Reward Function (R). At each decision step t , the agent selects assets to long and short. These positions are held for a fixed period of H_p steps. After closing the trades, we compute the profit or loss (PnL) for each asset individually and update the agent’s cash:

$$\text{cash}_{t+H_p} = \text{cash}_t + \sum_{i \in \mathcal{L}_t} \text{PnL}_i + \sum_{j \in \mathcal{S}_t} \text{PnL}_j \quad (6)$$

The return is then defined as the ratio of updated to current cash:

$$\text{return}_t = \frac{\text{cash}_{t+H_p}}{\text{cash}_t} \quad (7)$$

The reward signal is binary, based on whether the return is positive:

$$r_t = \begin{cases} +1, & \text{if } \text{return}_t > 1 \\ -1, & \text{otherwise} \end{cases} \quad (8)$$

This reward structure encourages the agent to make decisions that increase the portfolio’s value, while remaining robust to noisy return magnitudes by focusing on directional profitability.

Transition Dynamics & Learning Objective. In our setting, the environment’s transition dynamics $P(s_{t+1}|s_t, a_t)$ are unknown and inherently governed by the stochastic nature of the market. Rather than modeling these transitions explicitly, the agent interacts with the environment and learns a policy that maps observed states to actions based on observed outcomes.

The objective is to improve this policy through experience by maximizing the expected return over a finite horizon. Formally, the learning goal is to find a policy π that maximizes the expected cumulative reward:

$$J(\pi) = \mathbb{E}_\pi \left[\sum_{k=t}^T \gamma^{k-t} r_{k+1} \right] \quad (9)$$

where $\gamma \in [0, 1]$ is a discount factor that controls the agent’s preference for immediate versus future rewards, and r_{k+1} is the scalar reward received after executing the action at time step k .

This formulation enables the agent to refine its decision-making strategy by adjusting the policy to favor actions that lead to more profitable outcomes over time.

3.2 Deep Reinforcement Learning Architecture: Advantage Actor-Critic (A2C)

To solve the asset ranking problem, we employ the *Advantage Actor-Critic (A2C)* algorithm, a synchronous, on-policy variant of the actor-critic family (Mnih et al., 2016). A2C combines the stability of value-based methods with the flexibility of policy-based approaches. It consists of two neural networks:

- The **actor**, which learns a policy $\pi_\theta(a_t|s_t)$ to generate asset rankings based on the current state.

- The **critic**, which estimates the value function $V_\psi(s_t)$, providing a baseline for advantage estimation.

As illustrated in Figure 2, the full A2C framework includes both the network architecture and the training logic for learning from reward signals. The figure shows how shared feature extraction layers feed into two separate heads for policy and value estimation, and how gradients flow during training.

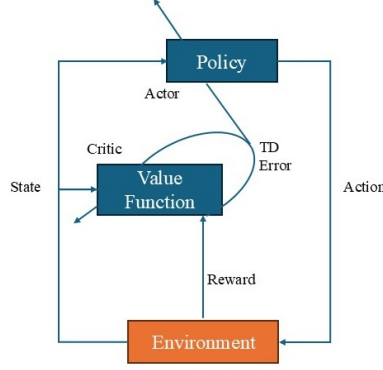


Figure 2: Advantage Actor-Critic (A2C) architecture: the agent consists of shared layers followed by separate policy (actor) and value (critic) output heads.

Network Architecture. The agent’s decision-making is powered by a deep neural network that processes the market state and outputs both a policy and a value estimate. The network begins with an input layer designed to accept the flattened and normalized state tensor, $s_t \in \mathbb{R}^{N \times H}$. This input is then fed into a shared feature extractor composed of a sequence of three fully connected (dense) layers. The first layer contains 128 units with a ReLU activation function, followed by a second layer of 64 units, also with ReLU activation. The final layer of the shared extractor has 32 units with a Sigmoid activation function, producing a compact, nonlinear representation of the market state. This shared representation is then passed to two separate output heads: a policy head (the actor), which generates the ranking probabilities; and a value head (the critic), which produces a scalar estimate of the state’s value.

Training Objective. The policy is trained using an n -step return formulation to reduce bias and variance in the advantage estimate:

$$G_t^n = \sum_{i=0}^{n-1} \gamma^i r_{t+i+1} + \gamma^n V_\psi(s_{t+n}) \quad (10)$$

$$A_t = G_t^n - V_\psi(s_t) \quad (11)$$

Actor Loss:

$$\mathcal{L}_{\pi_\theta} = -\mathbb{E}[\log \pi_\theta(a_t | s_t) A_t + \beta \mathcal{H}(\pi_\theta(\cdot | s_t))] \quad (12)$$

Critic Loss:

$$\mathcal{L}_{V_\psi} = \mathbb{E}[(G_t^n - V_\psi(s_t))^2] \quad (13)$$

The entropy bonus term $\mathcal{H}$ in the actor loss encourages exploration, with β controlling the

strength of this effect. This setup enables the agent to learn to rank cryptocurrencies based on sequential historical data, adjusting its strategy to maximize long-term profit while maintaining robust generalization to unseen market conditions.

3.3 Supervised Meta-Learning Filter: Adaptive Risk Management

While the DRL agent is designed to optimize ranking performance, its effectiveness can be compromised during periods of extreme market stress, a common feature of cryptocurrency markets. To address this, we introduce a risk management framework based on a suite of supervised machine learning models. This component functions as a meta-learner, providing a data-driven mechanism to dynamically gate trade execution by forecasting unfavorable volatility regimes.

3.3.1 Volatility-Based Signal Generation

The core of the filtering mechanism is the transformation of the DRL agent’s historical performance into a binary classification problem. This involves defining a target label based on market volatility and constructing a corresponding feature set from recent returns.

Simple volatility timing rules, while effective in traditional markets (Moreira & Muir, 2017), are often univariate and may fail to capture the complex, nonlinear nature of crypto risk regimes. This motivates our development of a more sophisticated, multivariate filter based on supervised meta-learning (Hospedales et al., 2021). The filter is a binary classifier trained to predict whether the DRL strategy’s own near-term volatility will exceed a predetermined threshold, effectively learning to identify adverse market conditions for the specific strategy it is managing.

However, this design introduces a potential for methodological circularity: the filter uses the strategy’s own historical returns as features to predict its future volatility. This could lead to pathological feedback loops; for instance, a series of losses could make the filter overly conservative, causing it to miss recovery opportunities. We mitigate this risk through several explicit design choices:

1. **Limited Memory:** The feature set for the meta-learner consists of a short window of past returns, preventing the model from developing an excessively long memory of past regimes.
2. **Threshold Robustness:** We test the framework’s sensitivity to the volatility threshold, ensuring that the filter’s performance is not contingent on a single, arbitrarily chosen parameter.
3. **Benchmark Comparison:** The filter’s performance is systematically compared against simpler, non-learning-based volatility timing rules to demonstrate its incremental value.

3.3.2 Volatility Signal Generation and Model Training

The volatility signal is the rolling standard deviation of the DRL strategy’s portfolio returns, σ_t , computed over a lookback window W_{vol} . The binary classification target, y_t^{vol} , is then generated

based on whether this signal exceeds a pre-determined threshold τ :

$$y_t^{vol} = \begin{cases} 1 & \text{if } \sigma_t < \tau \quad (\text{Low Volatility}) \\ 0 & \text{if } \sigma_t \geq \tau \quad (\text{High Volatility}) \end{cases} \quad (14)$$

The predictive features for the meta-model consist of a vector of the strategy’s most recent returns:

$$x_t = [r_{p,t-W_{feat}+1}, r_{p,t-W_{feat}+2}, \dots, r_{p,t}]^T \quad (15)$$

As confirmed by our implementation, we train and evaluate a comprehensive suite of classification models to identify the most robust predictor. This suite includes linear models (Logistic Regression), tree-based ensembles (Random Forest, Gradient Boosting), and advanced boosting methods (XGBoost, LightGBM, CatBoost). This rigorous evaluation ensures our results are not specific to a single model architecture and validates the overall meta-learning approach.

3.4 Portfolio Construction and Allocation

The final stage of our framework translates the DRL agent’s ranked signals and the meta-learner’s execution gate into risk-managed portfolios. This section provides the theoretical justification for our modular design and presents the formal optimization problems for each allocation methodology evaluated. At each rebalancing period, if the meta-learning filter permits a trade, we construct long-short portfolios by allocating 50% of capital to a long leg, comprising the top-ranked assets, and 50% to a short leg, comprising the bottom-ranked assets. The intra-leg capital allocation is then determined by one of several risk-based optimization schemes.

A core challenge in integrating machine learning with portfolio optimization is the potential for objective misalignment. Our DRL agent is trained to optimize a policy that maximizes cumulative binary rewards derived from an equally-weighted, cash-neutral portfolio. This objective, O_{RL} , is fundamentally different from the objectives of modern risk-based portfolio construction methods, O_{PC} . For example, the Minimum Variance objective is to minimize portfolio volatility, irrespective of expected returns. This creates a theoretical disconnect: the rankings produced by the agent are not explicitly optimized for the risk profiles targeted by the allocation methods. The misalignment error, ε_{align} , can be conceptualized as the performance degradation from using the RL-derived ranking, π_{RL}^* , within a portfolio construction method for which it was not explicitly trained.

Despite this misalignment, a modular approach offers compelling advantages over end-to-end systems that directly output portfolio weights, particularly in an institutional context.

1. **Transparency and Diagnostics:** A decoupled architecture allows for the independent analysis of signal quality (the ranking) and allocation efficiency. This is critical for risk management and regulatory compliance, as it avoids the “black box” problem where poor performance cannot be easily attributed to either faulty signals or flawed allocation.
2. **Mitigating Estimation Error:** The comprehensive analysis by DeMiguel et al. (2009) demonstrated that few sophisticated asset allocation models reliably outperform the naive

1/ N benchmark out-of-sample. This underperformance is driven almost entirely by the immense difficulty of accurately estimating expected returns (the first moment). By focusing on risk-based allocation methods, we explicitly sidestep this “error maximization” problem (Michaud, 1989). These methods rely only on the covariance matrix of returns (the second moment), which is substantially more stable and predictable than the mean.

3. **Computational Tractability:** Training a separate, complex DRL agent for each of the M distinct portfolio construction objectives is computationally prohibitive. Our modular approach is significantly more efficient, requiring a single DRL training phase followed by the application of multiple, computationally inexpensive optimization routines.

3.4.1 Covariance Matrix Estimation

The stability of any risk-based allocation method depends critically on the quality of the covariance matrix estimate, Σ . The sample covariance matrix is notoriously ill-conditioned, especially when the number of assets N is large relative to the length of the estimation window. To ensure our covariance matrix is robust, we employ the Ledoit-Wolf shrinkage estimator, which optimally combines the sample covariance matrix, S , with a more structured and stable target matrix, F .

$$\Sigma^{LW} = \delta^* F + (1 - \delta^*) S \quad (16)$$

where $\delta^* \in [0, 1]$ is the analytically derived optimal shrinkage intensity that minimizes the expected Frobenius norm between the estimator and the true covariance matrix.

3.4.2 Allocation Methodologies and Optimization Problems

At each rebalancing step t , the DRL agent provides a ranked list of assets. We form a long portfolio, $\mathcal{L}_t$, from the top K assets and a short portfolio, $\mathcal{S}_t$, from the bottom K assets of the ranked universe.. The following optimization methods are then applied independently to each leg to determine the weights, w , for the K assets within that leg. All methods require the weights to be non-negative ($w_i \geq 0$) and to sum to one ($\sum_{i=1}^K w_i = 1$), ensuring the capital in each leg is fully allocated.

Equal Weight (EW): The naive benchmark, which serves as a baseline for comparison.

$$w_i = \frac{1}{K} \quad (17)$$

Inverse Volatility (IV): A heuristic that weights assets inversely to their historical volatility, σ_i . This simple risk-based approach allocates less capital to more volatile assets.

$$w_i = \frac{\sigma_i^{-1}}{\sum_{j=1}^K \sigma_j^{-1}} \quad (18)$$

Minimum Variance (MinVar): Solves a quadratic program to find the portfolio with the lowest possible variance.

$$\min_w w^T \Sigma^{LW} w \quad (19)$$

Maximum Diversification (MD): Maximizes the portfolio’s diversification ratio, defined as the ratio of the portfolio’s weighted average of asset volatilities to its overall portfolio volatility (Choueifaty & Coignard, 2008).

$$\max_w \frac{w^T \sigma}{\sqrt{w^T \Sigma^{LW} w}} \quad (20)$$

where σ is the vector of asset volatilities.

Risk Parity (RP): Finds the unique portfolio where each asset contributes equally to the total portfolio risk. Unlike MinVar, it ensures that risk is balanced across all assets rather than concentrated in a few low-volatility assets (Maillard et al., 2010). The objective is to minimize the variance of the risk contributions:

$$\min_w \sum_{i=1}^K \left(w_i (\Sigma^{LW} w)_i - \frac{1}{K} w^T \Sigma^{LW} w \right)^2 \quad (21)$$

Minimum Conditional Value-at-Risk (MinCVaR): Optimizes to minimize the expected loss in the tail of the return distribution, providing a coherent measure of tail risk (Rockafellar & Uryasev, 2000). This is formulated as a linear program:

$$\min_{w, \zeta} \zeta + \frac{1}{1 - \alpha} \frac{1}{T_{hist}} \sum_{\tau=1}^{T_{hist}} \max(0, -w^T r_\tau - \zeta) \quad (22)$$

where ζ is the Value-at-Risk (VaR) at a confidence level α (set to 0.95), and r_τ are the historical return scenarios.

The integrated architecture of the proposed system operates sequentially at each rebalancing period. At each time step t , the DRL agent observes the current market state and generates a vector of scores, which are then used to rank the asset universe. Concurrently, the meta-learning filter assesses the strategy’s recent return stream to forecast the upcoming volatility regime. If the regime is classified as favorable, the ranked signals are passed to the portfolio construction module. Here, cash-neutral portfolios are formed by selecting assets from the top and bottom deciles for long and short positions, respectively, with capital allocated according to one of the risk-based optimization schemes. The resulting portfolio is held for one period, after which its performance determines the binary reward that is fed back to the DRL agent, thereby closing the learning loop.

Algorithm 1 RL-LTR

Require: Policy network π_θ with parameters θ_0 , Value function V_ψ with parameters ψ_0 , discount factor γ , learning rates α_θ and α_ψ , historical cryptocurrency dataset D , walk-forward split

- 1: Initialize: θ_0 , ψ_0 , and meta-learner M with initial parameters
- 2: **for** each episode $e \in E$ **do**
- 3: Split D into training set D_{train} and test set D_{test} for episode e
- 4: Initialize state s_0 (e.g., initial rankings or market conditions from D_{train})
- 5: **while** not converged in training **do**
- 6: **for** each time step t in D_{train} **do**
- 7: Sample action $a_t \sim \pi_\theta(a_t|s_t)$ (ranking scores for cryptocurrencies)
- 8: Partition cryptocurrencies into top and bottom percentiles based on a_t
- 9: Execute trades: long top percentile, short bottom percentile
- 10: Observe reward r_t : $r_t = 1$ if portfolio return > 0 , else -1
- 11: Observe next state s_{t+1}
- 12: Compute advantage: $A_t = r_t + \gamma V_\psi(s_{t+1}) - V_\psi(s_t)$
- 13: Update policy parameters θ : $\theta \leftarrow \theta + \alpha_\theta \nabla_\theta (\log \pi_\theta(a_t|s_t) A_t)$
- 14: Update value function parameters ψ : $\psi \leftarrow \psi - \alpha_\psi \nabla_\psi (G_t^n - V_\psi(s_t))^2$
- 15: **end for**
- 16: **end while**
- 17: Train meta-learner M on volatility of episode e to predict trade execution decisions
- 18: **end for**
- 19: Evaluate π_θ and M on D_{test}
- 20: **return** Trained policy π_θ , value function V_ψ , and meta-learner M

4 Empirical Framework and Results

4.1 Experimental Design and Setup

To rigorously evaluate the performance of our proposed framework, we construct an empirical setting that simulates realistic trading conditions and adheres to best practices for time-series analysis. This involves a precise definition of our dataset, a robust backtesting protocol designed to prevent look-ahead bias, and a comprehensive suite of performance metrics.

4.1.1 Data Description

The dataset was sourced from the Binance exchange API and comprises price data for the top 60 cryptocurrencies selected based on their average trading volume and market capitalization over the preceding three months as of each month. The data spans the period from January 1, 2020, to September 11, 2024. By choosing the top assets based on these criteria, we aimed to minimize look-ahead bias during the asset selection process, ensuring that our analysis is robust and reflective of current market conditions.

We utilize a 12-hour sampling frequency for our data. This choice provides a crucial balance between computational tractability and level of detail. For highly volatile and continuously traded assets like cryptocurrencies, higher frequency data captures short-term dynamics and market microstructures that are often missed by daily snapshots. This approach significantly increases the size of our dataset compared to using daily data, providing more observations to train our DRL agent and fostering a more robust learning process.

4.1.2 Data Pre-processing and Normalization

The raw price data used as input for our models exhibits substantial scale variations across different assets. For instance, the price of Bitcoin (BTC) is several orders of magnitude higher than that of other assets in the universe. Neural network models trained with gradient-based methods are known to be sensitive to the scale of input features; unscaled inputs can lead to unstable gradients and slow or non-convergent training.

To ensure numerical stability and prevent data leakage, we apply an asset-wise Min-Max normalization within each walk-forward fold. For each asset i , the normalization parameters are derived *only* from the prices within the current training data segment (P_i^{train}). These same $\min(P_i^{train})$ and $\max(P_i^{train})$ values are then used to normalize the prices in the corresponding one-month out-of-sample test set. This procedure simulates a realistic deployment scenario where the model is scaled using only historical information before being applied to new data.

The normalized price of asset i at time t , $p'_{i,t}$, is calculated as:

$$p'_{i,t} = \frac{p_{i,t} - \min(P_i^{train})}{\max(P_i^{train}) - \min(P_i^{train})} \quad (23)$$

where $p_{i,t}$ is the raw price of asset i at time t , and $\min(P_i^{train})$ and $\max(P_i^{train})$ are the minimum and maximum prices observed for that asset within the training segment only.

The asset-wise normalization approach encourages the DRL agent to focus on the *internal, temporal patterns* of each asset’s price trajectory rather than absolute price levels. This method mitigates bias from price magnitude, enabling the model to learn signals from the *shape* and *dynamics* of recent price history relative to each asset’s past. In contrast, a global normalization approach could skew learning toward the high-variance dynamics of larger assets. Our method allows for a fair evaluation based on each asset’s normalized momentum and mean-reversion characteristics, establishing a consistent basis for the agent’s ranking decisions.

4.1.3 Walk-Forward Validation

To maintain temporal integrity and mitigate look-ahead bias, we implement a walk-forward validation protocol with a rolling origin. The backtesting framework is organized into sequential folds, each comprising a four-month training period followed by a one-month out-of-sample testing phase. Upon completion of each evaluation, the time window advances by one month, and both the DRL agent and meta-learning models are fully retrained on the updated dataset prior to the next testing interval. A diagram illustrating this rolling methodology is provided in Appendix C (Figure C.1).

4.1.4 Performance Evaluation Metrics

To provide a comprehensive assessment of strategy performance, we utilize a suite of standard financial metrics that evaluate both absolute and risk-adjusted returns. The evaluation is based on the time series of the portfolio returns as defined in Equation (3) of our methodology. Let $\bar{r}_p$ be the mean and σ_p be the standard deviation of this series of portfolio returns over the out-of-sample test periods. The performance metrics are calculated as follows:

- **Cumulative Return:** The total growth of the initial capital over the entire backtest period.
- **Sharpe Ratio:** Measures the excess return per unit of total risk. The risk-free rate is assumed to be zero.

$$\text{Sharpe Ratio} = \frac{\bar{r}_p}{\sigma_p} \quad (24)$$

- **Sortino Ratio:** A modification of the Sharpe Ratio that penalizes only for downside volatility. It is calculated using σ_d , which represents the standard deviation of negative portfolio returns.

$$\text{Sortino Ratio} = \frac{\bar{r}_p}{\sigma_d} \quad (25)$$

- **Omega Ratio:** Compares probability-weighted gains above a return threshold (θ , set to zero) to losses below it:

$$\text{Omega Ratio} = \frac{\int_{\theta}^{\infty} (1 - F(r_p)) dr_p}{\int_{-\infty}^{\theta} F(r_p) dr_p} \quad (26)$$

- **Maximum Drawdown (MDD):** Represents the largest peak-to-trough decline in portfolio value.
- **Turnover:** Measures the frequency of trading, calculated as the average absolute change in portfolio weights $w_{i,t}$ for each asset i across the investment universe of size N over all time steps T .

$$\text{Turnover} = \frac{1}{T_{\text{total}}} \sum_{t=1}^{T_{\text{total}}} \sum_{i=1}^N |w_{i,t} - w_{i,t-1}| \quad (27)$$

- **Annualized Volatility (σ_p):** The standard deviation of the portfolio returns, scaled to an annual figure.

4.2 Model Implementation and Hyperparameters

Our experiments are conducted in Python, leveraging several open-source libraries for machine learning and reinforcement learning. The DRL agent is built using **Stable-Baselines3** (Raffin et al., 2021) with a PyTorch backend. The Learning-to-Rank benchmarks are implemented using PyTorch and **LightGBM** (Ke et al., 2017). The following sections detail the specific architectures and hyperparameters used.

4.2.1 DRL Agent Implementation

We employ the Advantage Actor-Critic (A2C) algorithm for our DRL agent. The agent’s policy and value functions are approximated by a multi-layer perceptron (MLP) architecture, as detailed in Section 3.2. The final hyperparameters for the A2C agent, selected to optimize performance during the initial training folds of our walk-forward validation, are presented in Appendix A.

The training process for the DRL agent is conducted within each four-month training fold of our walk-forward protocol. To foster the agent’s ability to learn a robust policy that accounts for

the sequential nature of market dynamics, we implement a continuous training approach through the entire training dataset for that fold. At each time step, the agent makes decisions, and the model parameters are updated based on the rewards received, as detailed in Algorithm 1.

This methodology contrasts with traditional episodic training, which may subject the agent to multiple randomized short periods. By utilizing the entire chronologically ordered dataset, we enable the agent to learn from the complete market trajectory, thereby capturing path-dependent phenomena and potential regime shifts inherent in historical data. The model trains for a predetermined number of timesteps corresponding to the length of the training data in each fold before being evaluated on the subsequent one-month out-of-sample testing period.

4.2.2 Benchmark and Filter Implementation

The benchmark models and meta-learning filters were implemented using a suite of standard libraries. For the supervised meta-learning filter, we used the default library parameters for the suite of classification models (e.g., Logistic Regression, CatBoost) as a robust baseline. For the LTR benchmarks, hyperparameters were tuned via a grid search methodology to maximize performance on a validation set within the initial walk-forward split. The full hyperparameter search space is detailed in Appendix A.

4.3 Comparative Analysis of Ranking Models

The initial stage of our empirical validation is designed to rigorously assess the efficacy of the proposed Deep Reinforcement Learning (DRL) agent as a signal generator. To this end, we conduct a comprehensive comparative analysis against a spectrum of established benchmarks. These benchmarks are selected to represent three distinct classes of ranking methodologies: (i) state-of-the-art supervised Learning-to-Rank (LTR) models, which are powerful but static in nature; (ii) conventional heuristic and classical quantitative strategies foundational to financial econometrics; and (iii) a random baseline to test for performance beyond statistical chance.

To ensure a direct and unbiased comparison of signal quality, all models are evaluated using a consistent portfolio construction methodology. At each 12-hour rebalancing interval, a cash-neutral, equally-weighted portfolio is formed. For all systematic strategies, this entails taking a long position in the top 5 ranked assets and a short position in the bottom 5. This standardized approach isolates the performance contribution of the ranking signal itself, abstracting away from the effects of more sophisticated allocation schemes, which are analyzed subsequently in Section 4.5.

The benchmark models include:

- **Supervised LTR Models:** We evaluate four prominent algorithms: LambdaMART, a gradient-boosted decision tree model; RankNet, a pairwise neural network approach; and two listwise neural network models, ListNet and ListMLE.
- **Classical & Heuristic Strategies:** We implement two benchmarks: Raw Returns (JT), the classical cross-sectional momentum (CSM) strategy, where assets are ranked on past cumulative returns (Jegadeesh & Titman, 1993); and Volatility Normalized MACD (Baz),

a more sophisticated technical heuristic that employs a trend estimator (MACD) normalized by asset volatility, a technique rigorously explored in momentum risk management literature (Barroso & Santa-Clara, 2015).

- **Baseline Model:** Random (Rand), a null benchmark where long and short portfolios are constructed from randomly selected assets, providing a baseline for performance attributable to chance.

The comparative performance results are presented in Table 2 and visualized in Figure 3.

Table 2: Performance of Ranking Models (Equal-Weight Portfolios)

Strategy	Cum. Return	Volatility	MDD	Sharpe Ratio	Sortino Ratio	Omega Ratio
DRL_rank	102.16%	25.10%	-28.73%	1.190	1.859	1.153
ListMLE	10.68%	44.72%	-38.64%	0.181	0.260	1.043
ListNet	-34.07%	42.88%	-49.08%	-0.638	-0.899	0.946
RankNet	-6.92%	46.08%	-45.62%	-0.116	-0.169	1.012
LambdaMart	-38.56%	44.75%	-59.45%	-0.697	-1.011	0.937
Raw Returns (JT)	-91.16%	63.69%	-92.88%	-0.932	-1.280	0.888
Vol Norm MACD (Baz)	-72.39%	42.90%	-76.95%	-0.885	-1.250	0.911
Random (Rand)	27.28%	39.60%	-40.44%	0.237	0.351	1.046



Figure 3: Cumulative Returns of All Ranking Strategies

The results yield a clear and compelling conclusion: the DRL agent delivers superior performance that is both economically and statistically significant. The agent achieved a cumulative return of 102.16% and a Sharpe Ratio of 1.19. Notably, this performance was realized with the lowest volatility (25.10%) and the smallest maximum drawdown (-28.73%) among all systematic strategies. This combination of high returns and robust risk management underscores the efficacy of the DRL agent’s dynamically learned, path-dependent policy.

Conversely, the analysis reveals a systemic failure across all static ranking paradigms. The supervised LTR models, despite their sophistication, proved unable to navigate the market dynamics, with most generating substantial losses. More striking is the breakdown of the classical quantitative strategies. The classical momentum strategy (Raw Returns (JT)) yielded highly negative returns (-91.16%), exhibiting extreme tail risk with a maximum drawdown of -92.88%. This result is a powerful illustration of the documented momentum crash phenomenon, confirming that such strategies are particularly fragile in the volatile cryptocurrency domain.

A particularly insightful result emerges from the Random (Rand) benchmark. Its positive cumulative return of 27.28% surpasses that of all sophisticated LTR and heuristic models. This finding implies that the signals generated by these static, systematic strategies were not merely uninformative but were actively detrimental, yielding performance worse than a naive random selection. The DRL agent was the only model to extract a genuine alpha-generating signal from the underlying noise, a conclusion starkly visualized by the divergent equity curves in Figure 3.

To understand why the DRL agent’s strategy is successful, we can visualize the structure of the ranking signals it produces. Figure 4 compares the signal heatmaps from the DRL agent against two representative static models, ListMLE and RankNet. Each cell compares the ranking of two assets. A deep red color means the model produces a strong signal to **long** the asset on the y-axis and **short** the asset on the x-axis. Deep blue indicates the opposite signal.

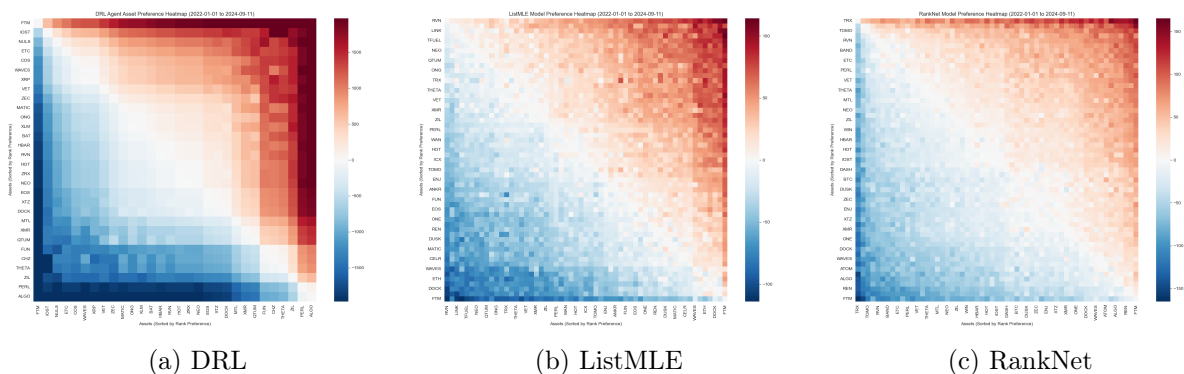


Figure 4: Comparative Analysis of Model Ranking Signals. Assets are sorted by their average rank. The DRL agent (a) generates clear, stable signals, while the static LTR models (b, c) produce noisy and conflicted signals.

The visual differences between the models are distinct. The DRL agent’s heatmap (a) displays a clear, block-like structure that distinctly separates assets chosen for the long portfolio (top-left, dark red) from those for the short portfolio (bottom-right, dark blue), indicating the agent has learned to generate strong, consistent long-short signals. In contrast, the heatmaps for ListMLE (b) and RankNet (c) are scattered and inconsistent, showing weak divisions between long and short candidates. This lack of stability is characteristic of the static models, which produce noisy and conflicting signals, undermining their effectiveness in a long-short strategy, as highlighted by their poor performance in Table 2. Thus, while the DRL agent demonstrates a stable policy yielding actionable signals, the static models fail to establish a coherent strategy.

4.4 Dissection of Framework Components: An Ablation Analysis

4.4.1 The Incremental Value of the Meta-Learning Filter

The second stage of our framework introduces an adaptive risk management filter designed to gate trade execution during periods of unfavorable volatility. To test its efficacy, we apply the suite of meta-learning filters to the raw return stream generated by our DRL ranking agent. The results, presented in Table 3 and Figure 5, demonstrate the significant impact of this risk management module.

The application of the filters universally improves the strategy’s risk-adjusted performance. The Logistic Regression filter, for example, increases the Sharpe Ratio from 1.19 to an impressive 2.11, while simultaneously reducing the maximum drawdown from -28.73% to -11.94%. Similarly, the Support Vector Machine filter achieves a Sharpe Ratio of 2.05 and reduces volatility to a low of 16.68%. The equity curves in Figure 5 illustrate this effect visually. While all filtered strategies (colored lines) follow the general upward trajectory of the unfiltered DRL agent, they exhibit markedly smoother paths and experience less severe drawdowns, particularly during the volatile periods in late 2023 and mid-2024. This provides strong empirical evidence that the meta-learning filter successfully identifies and avoids adverse market regimes, thereby enhancing the consistency and risk-adjusted profitability of the underlying DRL signal. Among the various classifiers tested, the advanced boosting methods (CatBoost, Gradient Boosting) and the linear model (Logistic Regression) appear to offer the most compelling trade-offs between return enhancement and risk mitigation.

Table 3: Performance of DRL Agent with Meta-Learning Filters

Model	Cum. Return	Ann. Volatility	MDD	Sharpe Ratio	Sortino Ratio	Omega Ratio
CatBoost	130.14%	17.69%	-13.41%	2.05	3.63	1.26
Gradient Boosting	135.80%	17.88%	-13.78%	2.09	3.73	1.26
K-Nearest Neighbors	87.30%	18.42%	-21.35%	1.42	2.38	1.25
LightGBM	108.32%	16.83%	-10.41%	1.86	3.30	1.33
Logistic Regression	161.71%	20.33%	-11.94%	2.11	3.64	1.55
Random Forest	155.93%	21.06%	-11.33%	1.98	3.51	1.39
Support Vector Machine	121.11%	16.68%	-13.97%	2.05	3.71	1.37
XGBoost	97.19%	16.58%	-14.32%	1.73	3.07	1.31

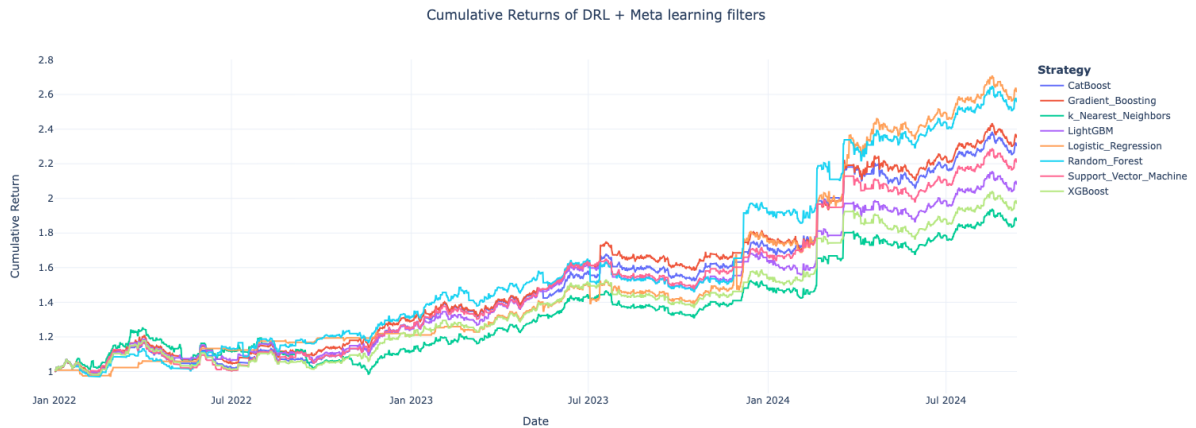


Figure 5: Cumulative Returns of DRL Strategy with Meta-Learning Filters

4.4.2 The Impact of Portfolio Allocation Methodology

The final stage of our framework involves translating the filtered DRL signal into a fully-weighted portfolio using sophisticated, risk-based optimization techniques. This analysis aims to quantify the benefit of moving beyond the naive Equal-Weight (EW) portfolio. We apply the six allocation methodologies to the return stream generated by the DRL agent with the best-performing meta-learning filter (Logistic Regression). The results, summarized in Table 4 and Figure 6, reveal that the choice of allocation method has a profound impact on both absolute and risk-adjusted returns.

The Maximum Diversification (MaxDiver) strategy emerges as the clear outperformer, achieving a cumulative return of 261.93% and a Sharpe Ratio of 2.85. This represents a significant improvement over the filtered Equal-Weight portfolio (168.76% return, 2.15 Sharpe), demonstrating the substantial value of optimizing for diversification. The equity curve for MaxDiver shows a distinctly superior growth trajectory. Other risk-based methods like Risk Parity and Inverse Volatility also consistently outperform the EW baseline on a risk-adjusted basis. These findings provide strong evidence for the third pillar of our framework: that the application of sophisticated, risk-based portfolio construction techniques can substantially enhance the alpha generated by an advanced machine learning signal.

Table 4: Performance of Portfolio Optimization Strategies with a Logistic Regression Filter

Method	Cum. Return	Ann. Volatility	MDD	Sharpe Ratio	Sortino Ratio	Omega Ratio
MinVar	150.45%	22.53%	-17.59%	1.80	2.93	1.34
MaxDiver	261.93%	21.43%	-9.98%	2.85	5.28	1.47
MinCVaR	98.08%	39.53%	-35.80%	0.73	1.14	1.28
EqualWeight	168.76%	20.61%	-12.50%	2.15	3.61	1.37
InverseVol	147.22%	18.15%	-9.36%	2.28	3.63	1.37
RiskParity	165.03%	18.86%	-9.34%	2.31	3.93	1.39

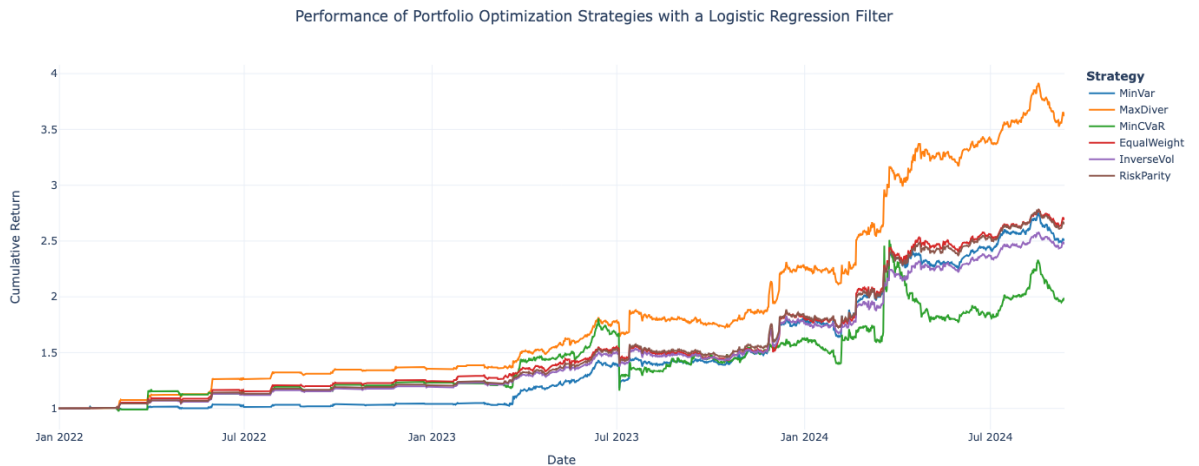


Figure 6: Performance of Portfolio Optimization Strategies with a Logistic Regression Filter

5 Discussion and Robustness

5.1 Discussion of Key Findings

Our empirical analysis yields three primary findings that advance the literature on Learning-to-Rank and dynamic asset allocation.

First, our results provide decisive evidence for the superiority of dynamic, path-dependent policies over static models in non-stationary and volatile financial markets. Our DRL-based ranker, which treats asset ranking as a sequential decision problem, achieved a Sharpe ratio of 1.19 and a cumulative return of 102.16%. In contrast, all static Learning-to-Rank benchmarks failed systematically—LambdaMART crashed to -38.56% returns, while traditional momentum strategies suffered catastrophic -91.16% losses. The explanation for this divergence lies in the agent’s learned behavior, as analyzed in Section 5.2. The key insight is that our agent successfully adapted its ranking strategy across market regimes, employing contrarian logic during low-volatility periods and switching to momentum-based ranking during high-volatility environments. This regime-adaptive ranking represents a fundamental advance over static LTR methods that cannot adjust to changing market dynamics.

Second, our modular architecture demonstrates significant practical advantages over end-to-end approaches. Rather than training a single black-box system, our three-stage framework—DRL ranker, meta-learning filter, and portfolio optimizer—enables transparent evaluation and risk control. The ablation analysis in Section 4 quantifies the value of each component. The meta-learning filter, which learns to gate trades based on the strategy’s own volatility, nearly doubled the Sharpe ratio to 2.11 (using a Logistic Regression classifier) while cutting the maximum drawdown in half. This modular design addresses a critical limitation of existing RL-based trading systems: the lack of interpretability and risk management controls that institutional investors require.

Third, we uncover a powerful synergy between data-driven rankings and modern portfolio construction techniques. While our DRL agent optimized a simple equal-weighted objective, applying Maximum Diversification allocation to its filtered rankings boosted the final Sharpe ratio to 2.85. This suggests that the DRL ranker, through its sequential learning process, implicitly identifies assets with favorable correlation structures that risk-based optimizers can explicitly exploit. This finding bridges machine learning and portfolio theory, showing that sophisticated ranking algorithms paired with principled allocation methods can generate substantial alpha.

5.2 Dynamic, Regime-Dependent Strategy Adaptation

The most compelling insight into the DRL agent’s success comes from analyzing its behavior across different market volatility regimes. We define three regimes (Low, Mid, High) based on the Deribit Bitcoin Volatility Index (DVOL) (Deribit, 2025).

To classify these periods, we use a quantile-based method on the historical distribution of the DVOL. The ‘Low Volatility’ regime corresponds to periods when the index was below its 33rd percentile, ‘High Volatility’ for periods above the 67th percentile, and ‘Mid Volatility’ for all periods in between. This data-driven approach ensures an objective and adaptive segmentation of market conditions. The analysis reveals that the agent did not learn a single, static strategy,

but rather a sophisticated, path-dependent policy that systematically adapts its approach to the prevailing market state.

As shown in Table 5 and Figure 7, in low- and mid-volatility regimes, the agent adopts a contrarian, mean-reversion strategy. It systematically selects long positions in assets that have exhibited negative recent momentum, positioning itself to capitalize on expected price reversals. Conversely, during high-volatility regimes, the agent fundamentally alters its strategy to a relative-value momentum approach. It selects long positions in assets with significantly higher positive momentum than its short positions. This demonstrates an ability to shift from betting against trends in calm markets to running with them during turbulent periods, a hallmark of an adaptive system.

Furthermore, the agent learned a sophisticated, asymmetric risk management policy. Across all regimes, it consistently constructs its short portfolio from assets with statistically lower volatility than its long portfolio. This learned heuristic mitigates the significant risks associated with short-selling in volatile markets. These adaptive patterns provide a clear explanation for the agent’s outperformance. By tailoring its strategy—shifting between contrarian and momentum approaches and actively managing risk based on the market context—the DRL agent successfully navigates the non-stationary dynamics that cause static models to fail. In the language of Operations Research, the agent’s policy is analogous to an optimal strategy for a regime-switching Markov Decision Process.

Table 5: Average Portfolio Characteristics by Volatility Regime

Regime	Portfolio Type	Momentum	Volatility	Corr. with BTC
High Volatility	Long	0.249***	0.0230***	0.389***
	Short	0.164	0.0186	0.417
Mid Volatility	Long	-0.018***	0.0279***	0.669***
	Short	-0.058	0.0296	0.606
Low Volatility	Long	-0.150*	0.0451***	0.689***
	Short	-0.120	0.0410	0.710

Notes: Stars denote significant differences between long and short portfolios (Wilcoxon rank-sum test: *** $p < 0.01$, ** $p < 0.05$, * $p < 0.1$).

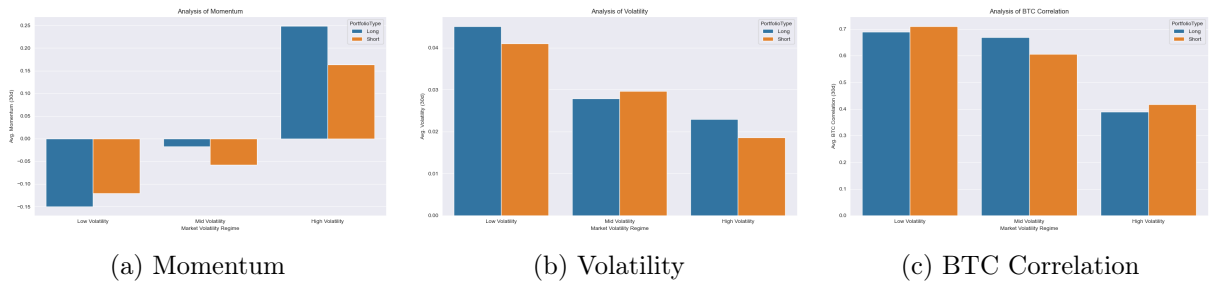


Figure 7: Analysis of DRL Agent’s Portfolio Characteristics Across Market Regimes. The agent systematically shifts its strategy based on market volatility, adopting a contrarian stance in low/mid volatility and a momentum stance in high volatility.

5.3 Persistent Portfolio Characteristics and Turnover

An analysis of the agent’s selections over the entire backtest period reveals persistent biases towards certain assets. The agent consistently favored long positions in assets such as FTM, IOST, and NULS, while persistently holding short positions in others like ALGO, PERL, and ZIL. A categorical analysis of the most persistently held assets indicates a thematic tilt: the long portfolio was heavily dominated by Layer-1 protocols, suggesting the agent learned to identify value in foundational blockchain infrastructure.

From an operational perspective, the strategy’s viability depends on transaction costs. We find that the strategy is active, with an average monthly portfolio turnover of 30.91%. While not insignificant, this is a manageable level for a high-frequency (12-hour rebalance) strategy. Given the substantial alpha generated (Sharpe Ratio of 2.85), this level of turnover does not preclude practical implementation, especially in the low-cost trading environment of digital asset markets.

5.4 Robustness Checks

We conducted extensive robustness tests to validate our findings across multiple dimensions, with full details provided in Appendix B.

- **Parameter Sensitivity:** We tested alternative design choices, including different rebalancing frequencies (6–24 hours) and portfolio sizes. While 12-hour rebalancing proved optimal, the strategy remained profitable across all tested frequencies. Similarly, though the symmetric 5-long/5-short configuration was optimal, alpha generation persisted across different portfolio sizes, with Sharpe ratios consistently exceeding 1.0 (see Tables B.1 and B.2).
- **Transaction Costs Impact:** acknowledging the strategy’s active nature (Section 5.3), the Sharpe ratio of our final, fully-integrated strategy (DRL + Filter + MaxDiver). decreased from 2.85 to a still-exceptional 2.56 (see Table 7). This confirms that the generated alpha is substantial enough to remain highly profitable after accounting for practical trading frictions.
- **Market Regime Analysis:** To test the agent’s adaptability, we analyzed its performance across the distinct market regimes within our out-of-sample period. This includes the severe 2022 bear market (‘crypto winter’) and the subsequent market recovery through 2024. The framework generated positive returns and strong risk-adjusted performance in all three sub-periods, providing powerful empirical evidence that the agent’s learned adaptive policy is effective across diverse market conditions.
- **Training Stability:** Deep reinforcement learning involves stochastic elements (e.g., random weight initialization) that can affect the final learned policy. To assess the framework’s sensitivity to stochastic initialization, we re-trained and backtested the DRL agent using four different random seeds, with results detailed in Appendix B. The analysis shows that while performance varies, the algorithm is robust, converging to a profitable policy (average Sharpe ratio of 0.84) in all cases.

For the main analysis presented in this paper, we followed a standard model selection protocol where the random seed was treated as a hyperparameter. We selected the seed (42) that yielded the best performance on the initial validation folds of our walk-forward analysis. This pre-specified model was then used for the entire subsequent out-of-sample evaluation. This procedure ensures the reported results are from a single, pre-selected model and avoids any selection bias on the final test data.

5.5 Volatility Threshold Sensitivity

A key parameter in our framework is the absolute volatility threshold, τ , which the meta-learning filter uses to gate trading. To ensure the strategy’s effectiveness is not overly sensitive to this choice, we re-evaluated the DRL strategy combined with the Logistic Regression filter across several thresholds.

The results, detailed in Table 6, confirm the robustness of the filtering mechanism. While the primary threshold of 0.009 provides the optimal Sharpe ratio of 2.11, the strategy delivers strong risk-adjusted performance across all tested values. Crucially, every configuration significantly improves upon the unfiltered DRL agent’s Sharpe Ratio of 1.19. This demonstrates that the value generated by the meta-learning filter is a robust feature of the framework, not an artifact of a single, fine-tuned parameter.

Table 6: Performance of DRL + Meta-Learning Filter with Different Absolute Volatility Thresholds

Volatility Threshold (τ)	Cum. Return	Volatility (Ann.)	Max Drawdown	Sharpe Ratio
0.007 (More Conservative)	141.35%	18.11%	-10.55%	2.04
0.009 (Primary)	161.71%	20.33%	-11.94%	2.11
0.011 (More Aggressive)	132.89%	22.45%	-16.81%	1.78
0.013	118.04%	24.18%	-21.02%	1.51

5.6 Transaction Costs Impact

To ensure practical viability, we simulated the impact of realistic trading frictions, acknowledging the strategy’s average monthly turnover of 30.91%. We applied a conservative cost of 5 basis points (0.05%) to each side of a trade on our final, fully-integrated strategy (DRL + Filter + MaxDiver).

As detailed in Table 7, the generated alpha remains substantial even after accounting for these costs. The Sharpe ratio decreased only moderately from a gross value of 2.85 to a net value of 2.56. Even with a higher cost of 10 basis points, the Sharpe ratio remains an impressive 2.32. This confirms that the strategy’s performance is not eroded by practical trading frictions and underscores the robustness of our integrated framework for real-world applications.

Table 7: Performance Net of Transaction Costs

Cost per Side	Cum. Return	Volatility (Ann.)	Max Drawdown	Sharpe Ratio
0 bp (Gross)	261.93%	21.43%	-9.98%	2.85
5 bp (0.05%)	228.31%	21.43%	-10.80%	2.56
10 bp (0.10%)	198.45%	21.43%	-11.50%	2.32

5.7 Limitations and Future Research

While our framework demonstrates significant promise, we acknowledge several limitations that open promising avenues for future work.

- **Limitations:** The DRL agent’s state is based solely on historical prices, ignoring other potentially valuable data sources. Moreover, our backtest assumes perfect execution at the closing price and does not model market impact or slippage.
- **Future Research:** A clear next step is to enrich the agent’s state representation with alternative data, such as on-chain metrics, order book information, or even natural language processing-derived sentiment scores. A second avenue involves developing more sophisticated, multivariate meta-learning filters that incorporate macroeconomic indicators to better anticipate market-wide stress events. Finally, applying advanced Explainable AI (XAI) techniques to the DRL agent could provide even deeper insights into its learned policy, moving beyond the behavioral analysis presented here to understand the specific features driving its decisions at any given moment.

6 Conclusion

This paper confronts the challenge of cross-sectional investing in the volatile and non-stationary cryptocurrency market. By formulating the problem as a sequential decision-making task addressed by a Deep Reinforcement Learning (DRL) agent, we move beyond the limitations of static ranking models. Our empirical results demonstrate that this dynamic, adaptive approach significantly outperforms traditional quantitative and machine learning strategies. Furthermore, we illustrate that embedding this DRL-based signal generator within a modular framework—complete with an adaptive risk-management filter and a sophisticated risk-based portfolio optimizer—enables the generation of substantial, robust, and practically achievable alpha.

The implications of our integrated framework extend beyond mere performance metrics; it provides a novel solution for navigating complex financial markets characterized by rapid changes and high uncertainty. In addition, it offers a transparent and controllable architecture for deploying advanced data-driven strategies within institutional settings, ensuring compliance with regulatory demands and risk management protocols. As financial markets increasingly integrate sophisticated technologies, our work paves the way for further research aimed at enhancing adaptivity in trading strategies.

In future work, we aim to explore the incorporation of regime detection techniques into our DRL framework. By detecting different market regimes, such as bullish, bearish, and sideways

trends, we can further enhance the adaptability of our investment strategies. This integration could enable the DRL agent to modify its approach according to current market conditions, thereby improving performance and risk management. Additionally, we envision investigating how various regime detection methodologies can complement our existing signal generation and portfolio optimization processes. This direction holds great potential for enhancing the robustness of our framework in navigating the intricate dynamics of volatile asset classes and could contribute significantly to future developments in both theory and practice.

References

- Ang, A., & Bekaert, G. (2002). International asset allocation with regime shifts. *The Review of Financial Studies*, 15(4), 1137–1187.
- Ardia, D., Bluteau, K., & Rüede, M. (2019). Regime changes in bitcoin garch volatility dynamics. *Finance Research Letters*, 29, 266–271.
- Barroso, P., & Santa-Clara, P. (2015). Momentum has its moments. *Journal of Financial Economics*, 116(1), 111–120.
- Borri, N. (2019). Conditional tail-risk in cryptocurrency markets. *Journal of Empirical Finance*, 50, 1–19.
- Burges, C., Shaked, T., Renshaw, E., Lazier, A., Deeds, M., Hamilton, N., & Hullender, G. (2005). Learning to rank using gradient descent. *Proceedings of the 22nd International Conference on Machine Learning*, 89–96.
- Burges, C. J. (2010). From ranknet to lambdarank to lambdamart: An overview. *Learning*, 11(23-581), 81.
- Cao, Z., Qin, T., Liu, T.-Y., Tsai, M.-F., & Li, H. (2007). Learning to rank: From pairwise approach to listwise approach. *Proceedings of the 24th International Conference on Machine Learning*, 129–136.
- Choueifaty, Y., & Coignard, Y. (2008). Toward maximum diversification. *The Journal of Portfolio Management*, 35(1), 40–51.
- Daniel, K., & Moskowitz, T. J. (2016). Momentum crashes. *Journal of Financial Economics*, 122(2), 221–247.
- DeMiguel, V., Garlappi, L., & Uppal, R. (2009). Optimal versus naive diversification: How inefficient is the 1/n portfolio strategy? *The review of Financial studies*, 22(5), 1915–1953.
- Deribit. (2025). Deribit bitcoin volatility index (dvol).
- Gu, S., Kelly, B., & Xiu, D. (2020). Empirical asset pricing via machine learning. *The Review of Financial Studies*, 33(5), 2223–2273.
- Hambly, B., Xu, R., & Yang, H. (2023). Recent advances in reinforcement learning in finance. *Mathematical Finance*, 33(3), 437–503.
- Hamilton, J. D. (1989). A new approach to the economic analysis of nonstationary time series and the business cycle. *Econometrica: Journal of the econometric society*, 357–384.
- Hospedales, T., Antoniou, A., Micaelli, P., & Storkey, A. (2021). Meta-learning in neural networks: A survey. *IEEE transactions on pattern analysis and machine intelligence*, 44(9), 5149–5169.

- Jegadeesh, N., & Titman, S. (1993). Returns to buying winners and selling losers: Implications for stock market efficiency. *The Journal of Finance*, 48(1), 65–91.
- Jiang, Z., Xu, D., & Liang, J. (2017). A deep reinforcement learning framework for the financial portfolio management problem. *arXiv preprint arXiv:1706.10059*.
- Joubert, J. F. (2022). Meta-labeling: Theory and framework. *The Journal of Financial Data Science*, 4(3), 31–49. <https://www.pm-research.com/content/ijjfds/4/3/31>
- Katsiampa, P. (2017). Volatility estimation for bitcoin: A comparison of garch models. *Economics Letters*, 158, 3–6.
- Ke, G., Meng, Q., Finley, T., Wang, T., Chen, W., Ma, W., Ye, Q., & Liu, T.-Y. (2017). Lightgbm: A highly efficient gradient boosting decision tree. *Advances in Neural Information Processing Systems* 30.
- Liu, T.-Y. (2011). *Learning to rank for information retrieval*. Springer.
- Liu, Y., & Tsyvinski, A. (2021). Risks and returns of cryptocurrency. *The Review of Financial Studies*, 34(6), 2689–2727.
- Lo, A. W. (2004). The adaptive markets hypothesis: Market efficiency from an evolutionary perspective. *Journal of Portfolio Management*, 30(5), 15–29.
- Maillard, S., Roncalli, T., & Teletche, J. (2010). The properties of equally weighted risk contribution portfolios. *The Journal of Portfolio Management*, 36(4), 60–70.
- Makarov, I., & Schoar, A. (2020). Trading and arbitrage in cryptocurrency markets. *Journal of Financial Economics*, 135(2), 293–319.
- Meyer, M., Barziy, I., & Joubert, J. F. (2023). Meta-labeling: Calibration and position sizing. *The Journal of Financial Data Science*, 5(2), 23–43. <https://www.pm-research.com/content/ijjfds/5/2/23>
- Michaud, R. O. (1989). The markowitz optimization enigma: Is 'optimized' optimal? *Financial Analysts Journal*, 45(1), 31–42.
- Mnih, V., Badia, A. P., Mirza, M., Graves, A., Lillicrap, T., Harley, T., Silver, D., & Kavukcuoglu, K. (2016). Asynchronous methods for deep reinforcement learning. *International Conference on Machine Learning*, 1928–1937.
- Moreira, A., & Muir, T. (2017). Volatility-managed portfolios. *The Journal of Finance*, 72(4), 1611–1644.
- Phillip, A., Chan, J. S., & Peiris, S. (2018). A new look at cryptocurrency volatility: A garch-midas approach. *Economics Letters*, 170, 31–35.
- Poh, D., Lim, B., Zohren, S., & Roberts, S. (2021). Building cross-sectional systematic strategies by learning to rank. *The Journal of Financial Data Science*, 3(2), 70–86. <https://doi.org/10.3905/jfds.2021.1.060>
- Poh, D., Lim, B., Zohren, S., & Roberts, S. (2022). Enhancing cross-sectional currency strategies by context-aware learning to rank with self-attention. *The Journal of Financial Data Science*, 4(3), 89–107. <https://doi.org/10.3905/jfds.2022.1.104>
- Raffin, A., Hill, A., Gleave, A., Kanervisto, A., Ernestus, M., & Dormann, N. (2021). Stable-Baselines3: A reliable implementation of reinforcement learning algorithms in python. *Journal of Machine Learning Research*, 22(268), 1–8. <http://jmlr.org/papers/v22/20-1364.html>

- Rockafellar, R. T., & Uryasev, S. (2000). Optimization of conditional value-at-risk. *Journal of Risk*, 2, 21–41.
- Rudin, C. (2019). Stop explaining black box machine learning models for high stakes decisions and use interpretable models instead. *Nature Machine Intelligence*, 1(5), 206–215.
- Schnaubelt, M. (2022). Deep reinforcement learning for the optimal placement of cryptocurrency limit orders. *European Journal of Operational Research*, 296(3), 993–1006.
- Sutton, R. S., & Barto, A. G. (2018). *Reinforcement learning: An introduction* (2nd). MIT Press.
- Vidal-Tomás, D. (2021). An investigation of cryptocurrency data: The market that never sleeps. *Quantitative Finance*, 21(12), 2007–2024.
- Xia, F., Liu, T.-Y., Wang, J., Zhang, W., & Li, H. (2008). Listwise approach to learning to rank: Theory and algorithm. *Proceedings of the 25th International Conference on Machine Learning*, 1192–1199.

Appendix

Appendix A. Hyperparameter Tuning for Benchmark Models

This appendix details the hyperparameter search space used for tuning the machine learning-based benchmark models. The final parameters were selected based on the highest validation performance on the initial walk-forward split and were then held constant for the remainder of the backtest to prevent look-ahead bias.

Table A.1: Hyperparameter Search Space for Benchmark Models

Model	Hyperparameter	Search Space / Value
RankNet, ListNet, ListMLE	Learning Rate	{0.001, 0.005, 0.01}
	Hidden Layer Size	{16, 32, 64}
	Number of Epochs	{30}
	Batch Size	{32, 64}
	Early Stopping Patience	{7}
LambdaMART	Learning Rate	{0.02, 0.05, 0.1}
	N_Estimators (Trees)	{100, 200}
	Max Leaf Nodes	{7, 10}
	Min Samples Leaf	{32, 64}
	Metric	NDCG@10

A.1 DRL Agent Hyperparameters

Table A.2: DRL Agent (A2C) Final Hyperparameters

Hyperparameter	Value	Description
Learning Rate ($\alpha_\theta, \alpha_\psi$)	1e-5	The learning rate for the Adam optimizer.
Discount Factor (γ)	0.99	The factor for discounting future rewards.
Entropy Coefficient (β)	0.01	The weight of the entropy bonus to encourage exploration.
n_steps	5	The number of steps to run for each environment per update.
Value Function Coefficient	0.5	The weight of the value function loss in the total loss.
Max Gradient Norm	0.5	The maximum value for gradient clipping.

Appendix B. Robustness Check Details

This appendix provides the detailed empirical results for the robustness checks summarized in Section 5.

B.1 Parameter Sensitivity Analysis

To validate that our framework’s performance is not contingent on a specific set of design choices, we tested its sensitivity to variations in rebalancing frequency and portfolio size. Table B.1 com-

compares the performance of the baseline DRL agent across different rebalancing intervals. Table B.2 shows the performance with different numbers of assets in the long and short portfolios.

Table B.1: Performance Across Different Rebalancing Frequencies

Frequency	Cum. Return	Volatility (Ann.)	Max Drawdown	Sharpe Ratio
6-Hour	48.43%	16.87%	-16.28%	0.97
12-Hour	102.16%	25.10%	-28.73%	1.19
24-Hour	68.92%	17.87%	-11.48%	0.91

Table B.2: Performance Across Different Portfolio Sizes (K)

Configuration	Cum. Return	Volatility (Ann.)	Max Drawdown	Sharpe Ratio
3-Long / 3-Short	48.45%	21.07%	-24.30%	0.58
5-Long / 5-Short	102.16%	25.10%	-28.73%	1.19
6-Long / 6-Short	105.24%	16.91%	-21.03%	0.96
3-Long / 6-Short	90.95%	21.30%	-24.03%	0.73
6-Long / 3-Short	126.36%	20.86%	-31.10%	0.91

B.2 Training Stability Verification

To confirm that the results are not an artifact of stochasticity in the DRL training process, we re-trained and backtested the primary DRL agent using four different random seeds. The results in Table B.3 show that while performance varies, the algorithm reliably converges to a profitable policy.

Table B.3: Performance Across Different Random Seeds

Random Seed	Cum. Return	Volatility (Ann.)	Max Drawdown	Sharpe Ratio
42 (Primary)	102.16%	25.10%	-28.73%	1.19
1	75.82%	19.26%	-19.03%	0.85
1234	50.71%	19.58%	-26.70%	0.63
4321	38.14%	13.31%	-18.60%	0.69
<i>Mean</i>	<i>66.71%</i>	<i>19.31%</i>	<i>-23.27%</i>	<i>0.84</i>
<i>Std. Dev.</i>	<i>28.48%</i>	<i>4.78%</i>	<i>5.01%</i>	<i>0.25</i>

Appendix C. Walk-Forward Validation Scheme

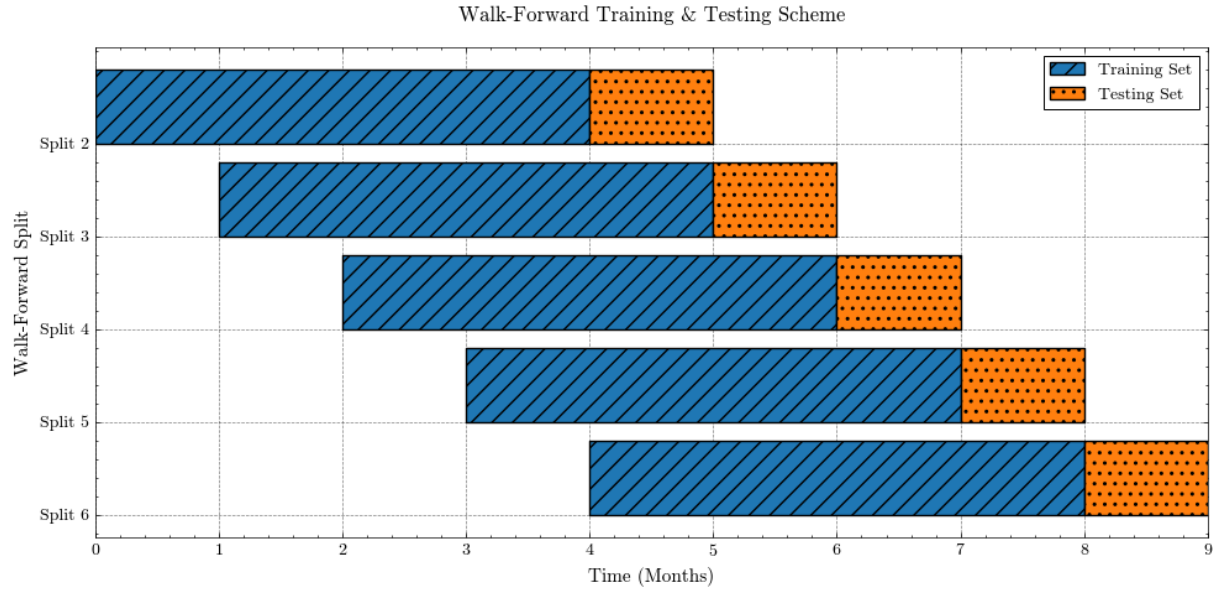


Figure C.1: Walk-Forward Training & Testing Scheme. The diagram illustrates the rolling origin approach, where each subsequent fold incorporates the latest training data while maintaining temporal integrity.